MEMORIA PROYECTO FINAL BBDD

Integrantes: Ángela Porres Cobb y Míriam Mingo Revuelta

Curso: 2º B iMAT

Primer apartado

(Miriam)

Para esta primera parte combinaremos una base de datos relacional (MySQL) y una base de datos no relacional (MongoDB), con el objetivo de aprovechar las ventajas de ambos modelos.

Por un lado, en MySQL hemos almacenado los datos que presentan una estructura claramente definida y relaciones bien establecidas entre entidades. En concreto, hemos definido tablas para usuarios, productos y reviews. La tabla de usuarios almacena el identificador único del usuario (reviewerID) junto con su nombre (reviewerName). La tabla de productos almacena el identificador del producto (asin) y su categoría (videojuegos, juguetes, etc.). Finalmente, la tabla de reviews contiene la información de cada reseña, incluyendo la puntuación (overall), el tiempo en formato Unix (unixReviewTime) y la fecha de la review (reviewTime), además de las claves foráneas que relacionan cada review con su usuario y su producto. (El uso de claves primarias y foráneas garantiza la integridad referencial, evitando inconsistencias en los datos.)

Por otro lado, en MongoDB hemos almacenado los campos menos estructurados y de mayor tamaño, concretamente el texto completo de la review (reviewText), el resumen (summary) y la información de utilidad (helpful). Estos datos presentan una estructura flexible y no requieren relaciones directas con otras entidades, por lo que encajan mejor en un modelo basado en documentos. Esto permite mejorar la escalabilidad del sistema, ya que los textos pueden ser largos y variables, evitando sobrecargar la base de datos relacional con información que no se beneficia de su modelo estructurado. Además, MongoDB permite una inserción más flexible y eficiente de este tipo de datos.

En resumen, se ha diseñado un sistema híbrido donde MySQL gestiona la información estructurada y relacional, mientras que MongoDB se encarga de los datos textuales y menos estructurados, logrando así un equilibrio entre consistencia, eficiencia y escalabilidad.

ESQUEMA RELACIONAL

USUARIOS (reviewerID, reviewerName)

PRODUCTOS (asin, categoría)

REVIEWS (id_review, reviewerID, asin, overall, unixReviewTime, reviewTime)

reviewerID FK REFERS TO USUARIOS(reviewerID)

asin FK REFERS TO PRODUCTOS(asin)

Para diseñar el esquema relacional separamos las entidades principales del sistema: usuarios, productos y reviews.

La tabla de usuarios almacena información única de cada usuario, utilizando reviewerID como clave primaria. De este modo, se evita duplicar información de usuarios cuando estos realizan múltiples reviews.

La tabla de productos almacena los artículos identificados por su asin, junto con la categoría a la que pertenecen. Esto permite clasificar los productos y facilita consultas posteriores filtrando por tipo de producto.

La tabla de reviews actúa como entidad intermedia que relaciona usuarios y productos. En ella se almacenan las valoraciones (overall), así como la información temporal (unixReviewTime y reviewTime). Definimos una clave primaria artificial (id_review) para identificar de manera única cada review y claves foráneas (reviewerID y asin) que garantizan la integridad referencial con las tablas de usuarios y productos.

Este diseño sigue una estructura normalizada, evitando redundancias y facilitando la eficiencia en las consultas. Además, permite representar correctamente las relaciones de tipo uno a muchos entre usuarios y reviews y entre productos y reviews.

Segundo apartado

(Miriam)

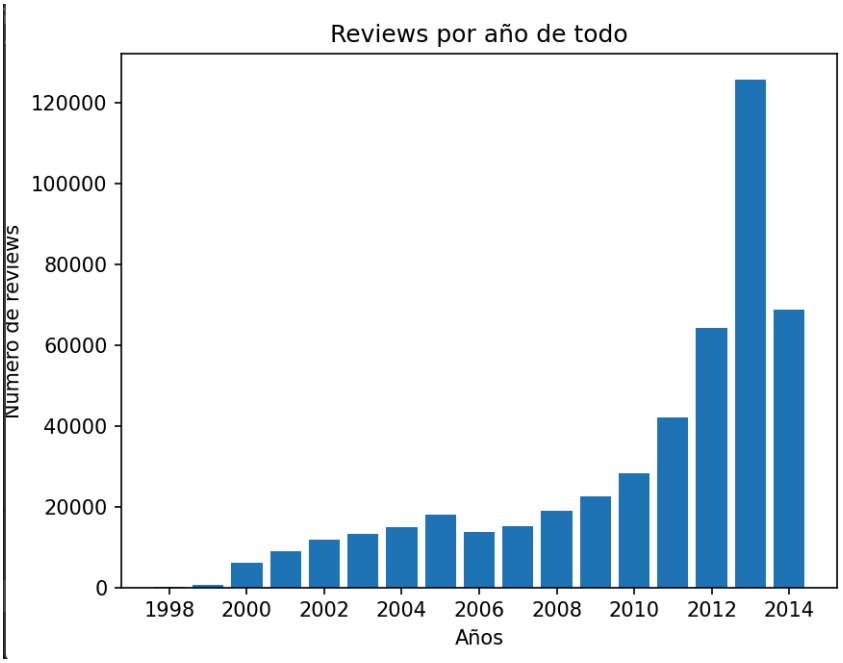
Para la visualización de los datos hemos utilizado la librería matplotlib para la representación gráfica y la librería wordcloud para la generación de nubes de palabras.

El fichero se basa en un menú interactivo por terminal que permite al usuario seleccionar diferentes tipos de gráficos para analizar la información almacenada en las bases de datos MySQL y MongoDB.

Dependiendo de la gráfica, se ha permitido al usuario elegir si desea visualizar los datos de todos los productos, filtrarlos por una categoría concreta o por un producto específico. Además, el programa incluye validaciones para comprobar que la categoría o el producto introducidos existen realmente en la base de datos, evitando así errores y garantizando la consistencia de los resultados mostrados.

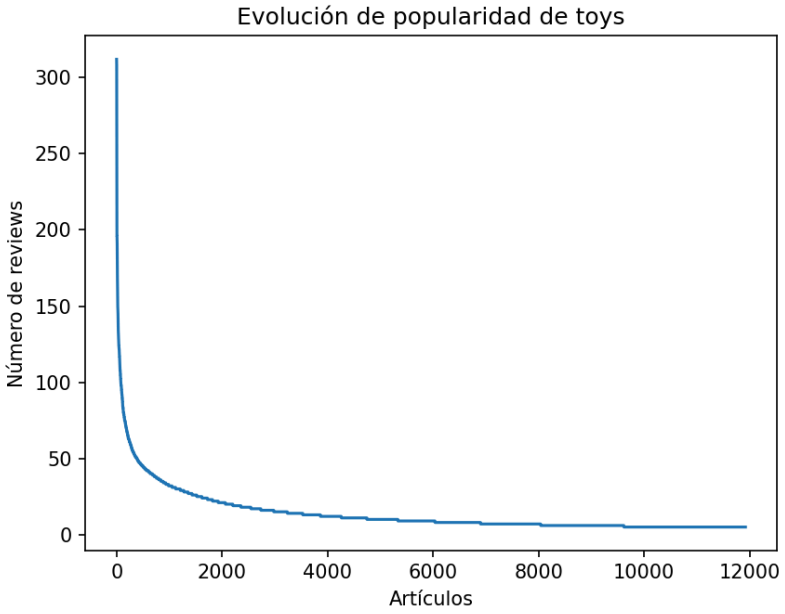
1. Evolución de reviews por años

   Para esta gráfica, el enunciado pide representar el número de reviews por año, por lo que es necesario agrupar los datos en función del año en el que se realizó cada review. Dado que la fecha está almacenada en el campo reviewTime, se utiliza la función YEAR() para extraer el año y posteriormente se aplica un GROUP BY sobre este valor para contar cuántas reviews corresponden a cada año. En el caso de querer filtrar por categoría, añadimos un INNER JOIN con la tabla de productos para acceder a ese atributo.



2. Evolución de la popularidad de los artículos

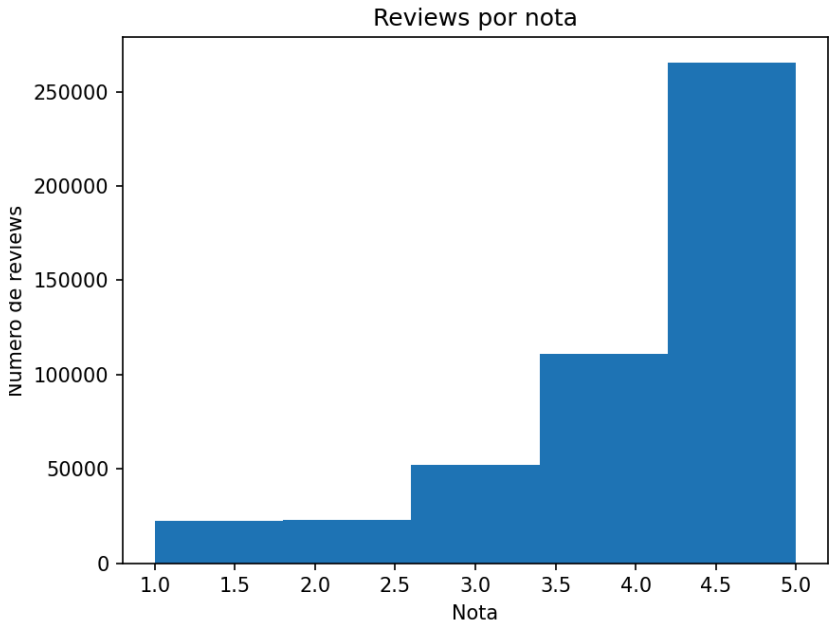
El enunciado define la popularidad como el número de reviews por producto, por lo que la consulta debe agrupar las reviews por identificador de producto (asin) y contar cuántas tiene cada uno. Para ordenar los productos de mayor a menor popularidad, se utiliza ORDER BY COUNT(*) DESC. El uso del INNER JOIN permite filtrar por categoría en caso de que el usuario no quiera visualizar todos los productos.



3. Histograma por nota

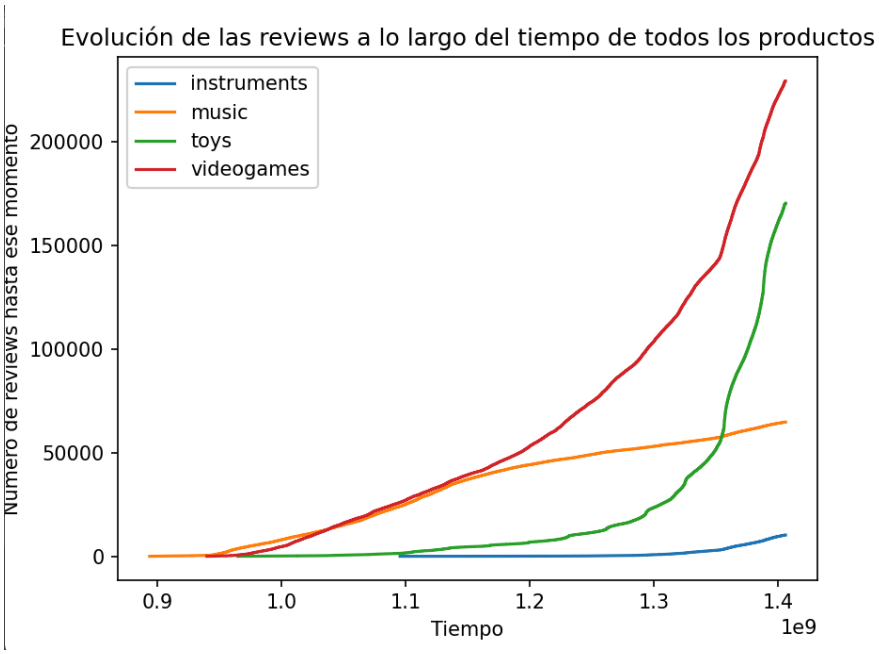
Para construir el histograma de notas, se necesita obtener todas las puntuaciones (overall) de las reviews. Dependiendo de la opción elegida por el usuario, la consulta

puede filtrar por categoría (que requiere un INNER JOIN con productos), por producto específico (asin) o no aplicar ningún filtro.



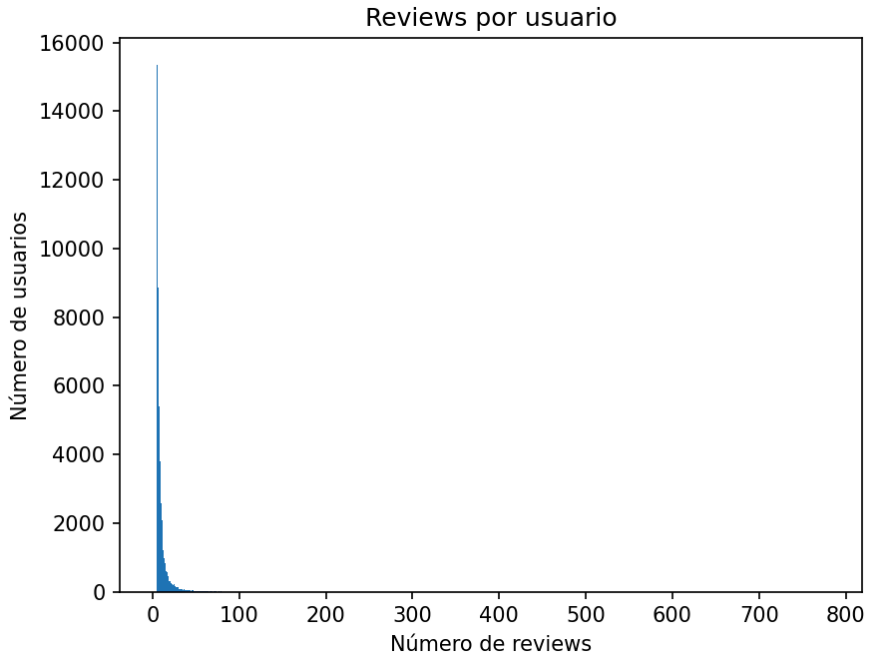
4. Evolución de las reviews a lo largo del tiempo para todas las categorías

Para mostrar la evolución acumulada de reviews a lo largo del tiempo diferenciada por categoría, obtenemos el campo unixReviewTime ordenado cronológicamente, junto con la categoría mediante un INNER JOIN. Posteriormente, construimos una acumulación progresiva de reviews para cada categoría en Python, generando una curva independiente por cada una. Así conseguimos ver cómo crecen las reviews a lo largo del tiempo.



5. Histograma de reviews por usuario

Agrupamos las reviews por reviewerID (cada usuario tiene su id) y contamos cuántas hay en cada grupo. El resultado es una lista de cantidades que posteriormente se representa como un histograma.



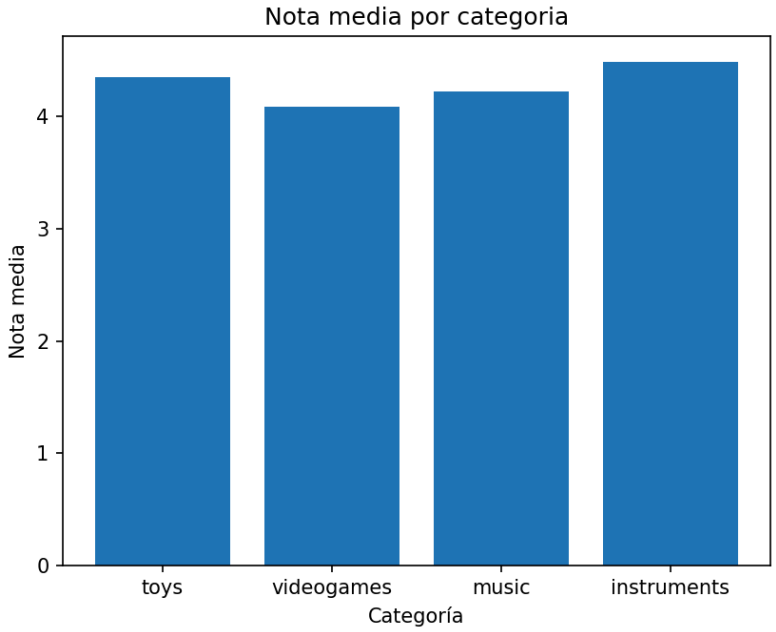
6. Nube de palabras

Aunque la nube de palabras debe generarse a partir del campo summary (almacenado en MongoDB), el filtrado por categoría se realiza en MySQL, por lo que primero obtenemos los identificadores de las reviews correspondientes a dicha categoría mediante un INNER JOIN. Posteriormente, accedemos a MongoDB para recuperar el texto asociado a cada review y construimos un conjunto de palabras, filtrando aquellas con longitud menor o igual a tres caracteres (para eliminar conectores).



7. Gráfica adicional: Nota media por categoría

   Como gráfica adicional hemos decidido realizar un diagrama de barras de la nota media por categoría. Esta representación permite comparar de forma clara la valoración media entre diferentes tipos de productos y ver cuáles tienen mayor éxito entre consumidores. Agrupamos las reviews por categoría y aplicamos la función de agregación AVG() sobre el campo overall, además usamos un INNER JOIN para relacionar cada review con su categoría correspondiente.



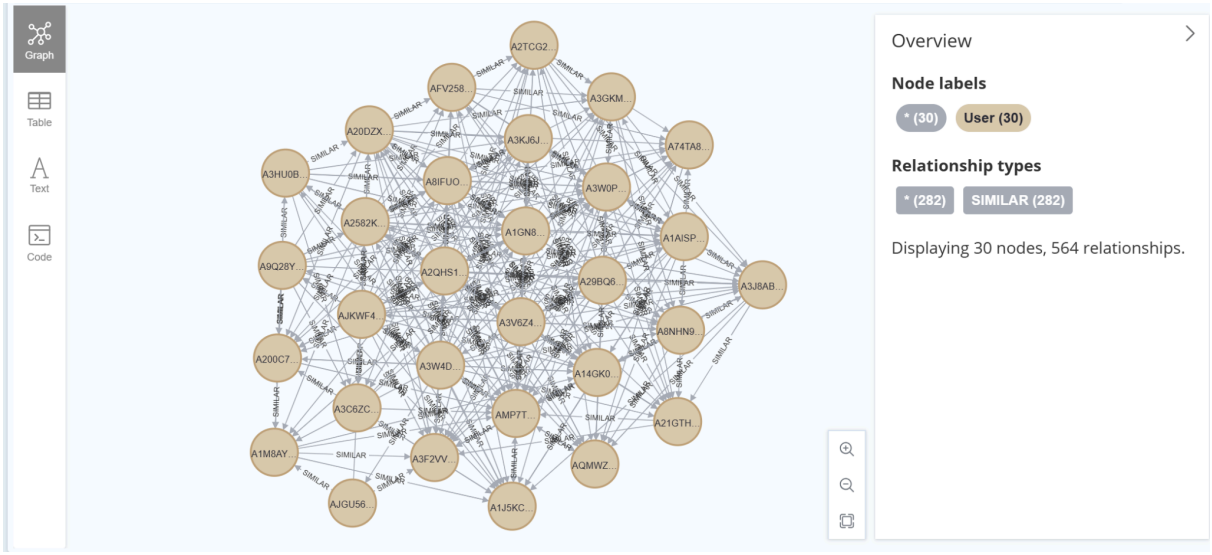
Tercer apartado

(Ángela)

Esta parte del proyecto consiste en desarrollar un módulo en Python que permite generar distintos grafos en Neo4J a partir de los datos almacenados en MySQL. El programa establece conexiones tanto con MySQL como con Neo4J, permitiendo extraer información relacional y transformarla en estructuras de grafo.

3.1 Obtener similitudes entre usuarios y mostrar los enlaces en Neo4J

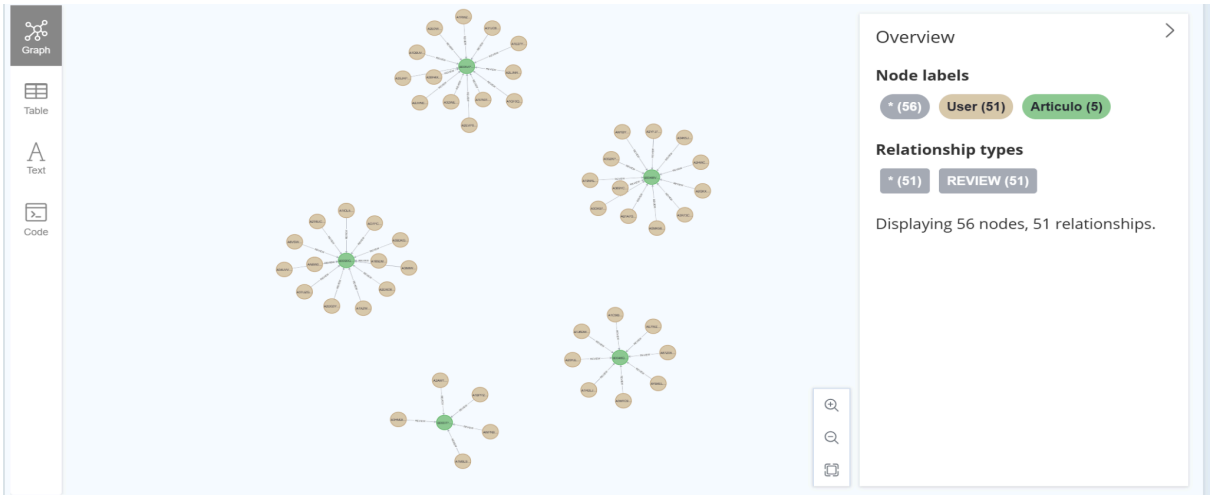
En esta funcionalidad hemos seleccionado los 30 usuarios con mayor número de reviews desde MySQL. Para cada pareja de usuarios calculamos la similitud de Pearson utilizando únicamente los artículos valorados en común. Posteriormente, creamos un grafo en Neo4J donde cada nodo representa un usuario y cada arista representa la similitud entre dos usuarios. Finalmente, realizamos una consulta en Neo4J para obtener el usuario con mayor número de vecinos.



Hemos construido un grafo en Neo4J donde cada nodo representa un usuario y cada arista representa la similitud entre usuarios calculada mediante la correlación de Pearson. En la figura se muestran las relaciones entre los 30 usuarios con mayor número de reviews.

3.2 Obtener enlaces entre usuarios y artículos

En este apartado, solicitamos al usuario que introduzca una categoría y el número de artículos que desea visualizar. A partir de la categoría seleccionada, obtenemos todos los artículos disponibles en la base de datos y seleccionamos de forma aleatoria una muestra de dichos artículos. Para cada uno de estos artículos, recuperamos las reviews asociadas, incluyendo la información de los usuarios que los han valorado y, por último, construimos un grafo en Neo4J en el que representamos nodos de tipo *User* y nodos de tipo *Articulo*, conectados mediante relaciones *REVIEW*, que reflejan las interacciones entre usuarios y artículos. Cada relación almacena como propiedades la puntuación (nota) y el instante temporal (tiempo).



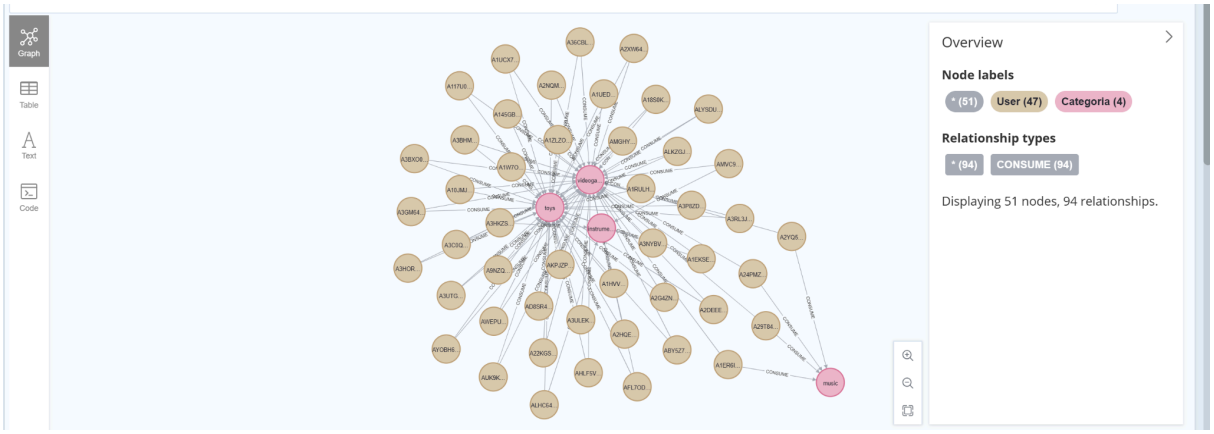
En el grafo generado en Neo4J, cada nodo representa un usuario o un artículo, y las relaciones representan las reviews realizadas, incluyendo como propiedades la nota y el instante temporal de la misma.

3.3 Obtener algunos usuarios que han visto más de un determinado tipo de artículo

En este apartado hemos seleccionado los primeros usuarios ordenados por su nombre (reviewerName). Sin embargo, para el procesamiento interno hemos utilizado el identificador reviewerID, ya que es único y así evitamos problemas derivados de nombres duplicados o vacíos.

Para cada usuario:

1. Se calcula cuántas reviews ha realizado en cada categoría.
2. Se seleccionan únicamente aquellos usuarios que han puntuado artículos de más de una categoría.
3. Se genera un grafo en Neo4J con:
    a. nodos User
    b. nodos Categoria
    c. relaciones CONSUME

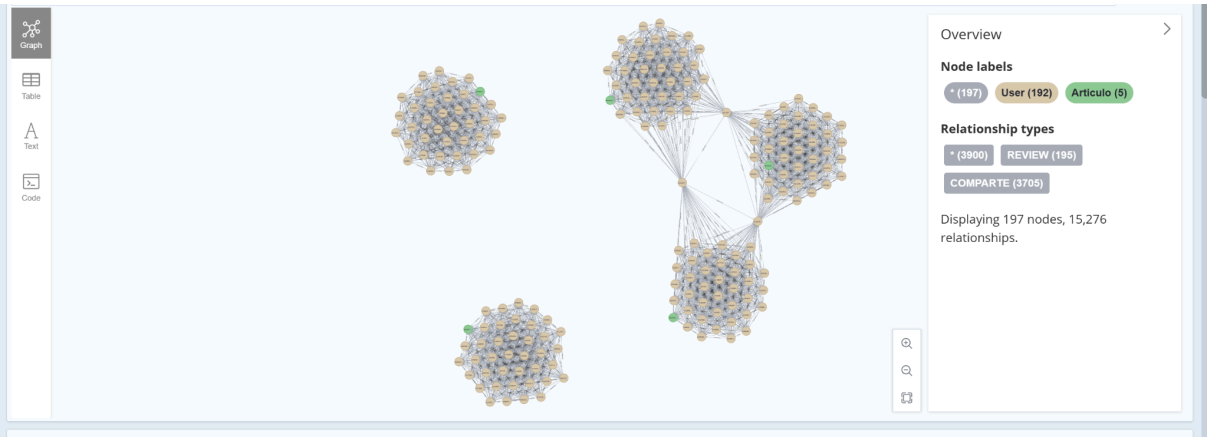


En Neo4J se han representado nodos de usuario y nodos de categoría, unidos mediante relaciones *consume*, cuya propiedad total indica el número de artículos puntuados por el usuario en cada categoría.

3.4 Artículos populares y artículos en común entre usuarios

En este apartado hemos seleccionado los 5 artículos más populares que cuentan con menos de 40 reviews. Para ello, en primer lugar hemos agrupado las reviews por artículo (asin) y hemos calculado el número de valoraciones de cada uno. A continuación, hemos filtrado aquellos artículos con menos de 40 reviews y, dentro de este conjunto, hemos seleccionado aquellos con mayor número de valoraciones

Posteriormente, hemos identificado los usuarios que han puntuado dichos artículos. Con esta información, hemos generado un grafo en Neo4J en el que representamos los usuarios y los artículos, conectados mediante relaciones *REVIEW*. Además, hemos creado relaciones adicionales entre usuarios de tipo *COMPARTE*, que indican el número de artículos que han valorado en común, lo que nos permite analizar la similitud entre ellos.



Cuarto apartado

(Ángela)

Para la cuarta parte del proyecto hemos desarrollado un script llamado inserta_dataset.py, cuyo objetivo es insertar un nuevo conjunto de datos, distinto de los utilizados previamente, en el sistema. En nuestro caso, hemos seleccionado **Books_5.json** (fichero del conjunto original de Amazon Reviews) cuya ruta y categoría asociada están definidos en el fichero de configuración.

Hemos utilizado el mismo procedimiento que en la primera parte del proyecto, manteniendo la separación entre datos estructurados y no estructurados. La información estructurada de cada review se almacena en MySQL, incluyendo el identificador del usuario (reviewerID), el identificador del producto (asin), la categoría, la puntuación (overall), el instante temporal (unixReviewTime) y la fecha (reviewTime).

Por otro lado, la información textual y menos estructurada, como el contenido de la review (reviewText), el resumen (summary) y la utilidad (helpful), se almacena en MongoDB.

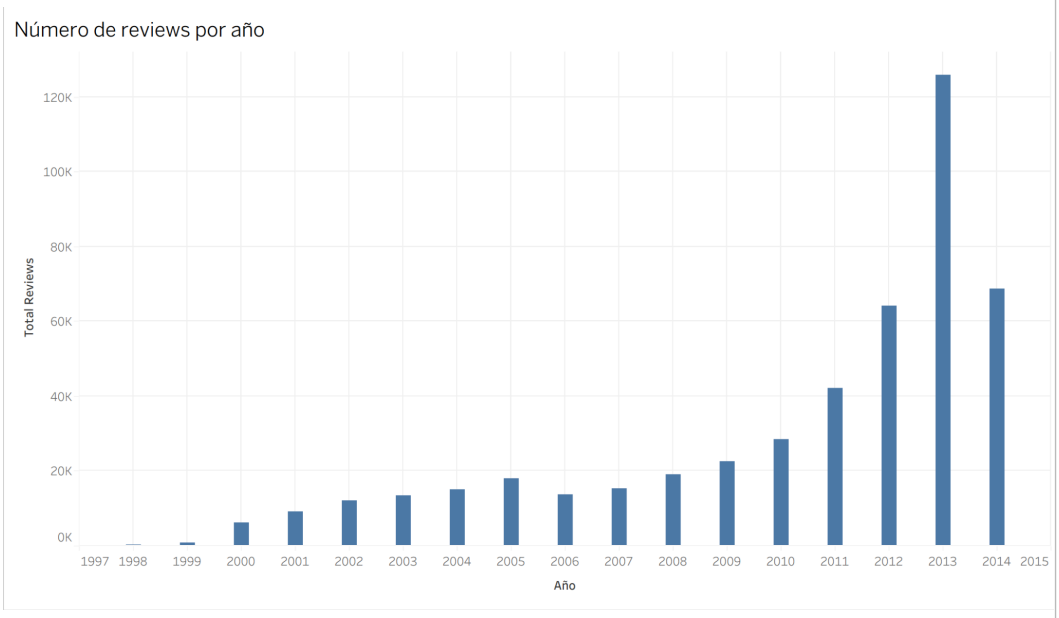
Para evitar duplicar información, utilizamos inserciones con INSERT IGNORE en las tablas de usuarios y productos, lo que nos permite que un mismo usuario pueda haber realizado reviews en distintas categorías sin generar errores.

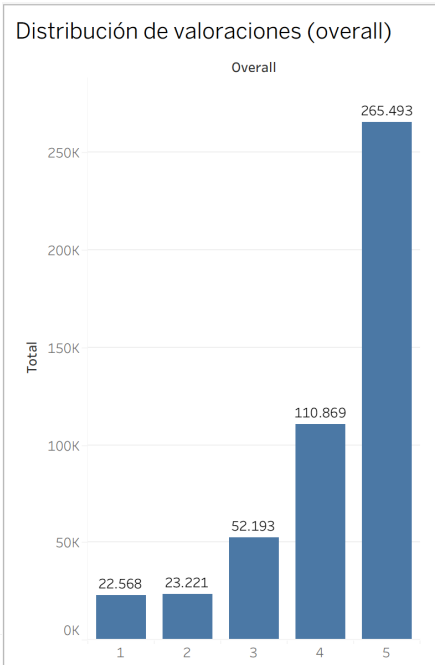
Tras la inserción de las nuevas reviews en MySQL, recuperamos los identificadores generados automáticamente en la tabla reviews, que utilizamos para asociar cada documento almacenado en MongoDB mediante el campo review_id. De esta forma, mantenemos la relación entre ambas bases de datos. Finalmente, realizamos el proceso leyendo el fichero de entrada línea a línea, evitando cargar el dataset completo en memoria.

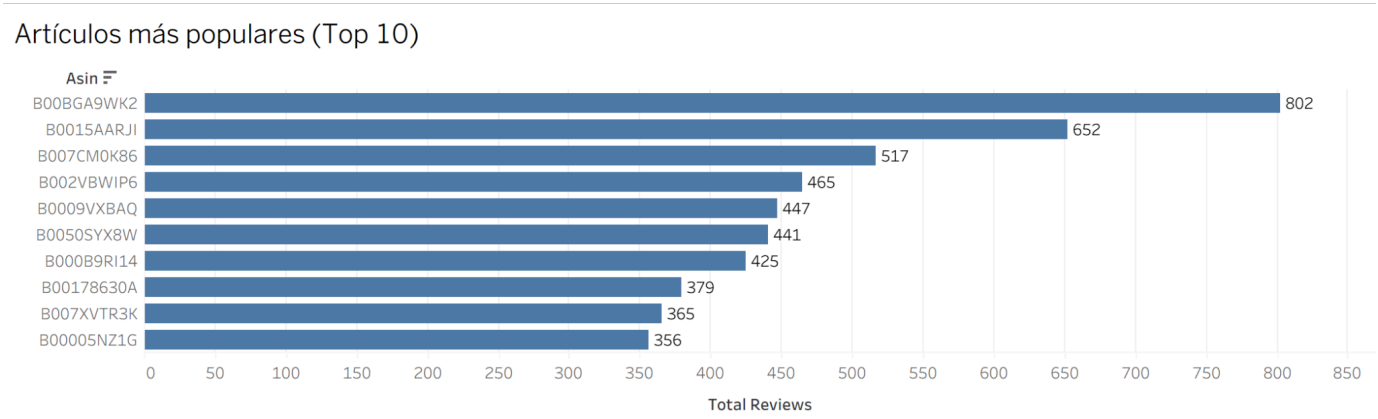
Quinto apartado:

(Ángela)

1. Gráficas mostradas anteriormente utilizando Tableau







Las figuras anteriores muestran tres visualizaciones relevantes obtenidas a partir de los datos almacenados en la base de datos. En primer lugar, representamos la evolución del número de reviews a lo largo del tiempo, observando un crecimiento progresivo en la cantidad de valoraciones realizadas por los usuarios. En segundo lugar, mostramos la distribución de las puntuaciones (overall), donde apreciamos que la mayoría de las valoraciones se concentran en puntuaciones altas, especialmente en 4 y 5 estrellas, lo que indica una tendencia positiva en las opiniones de los usuarios. Por último, graficamos los artículos más populares, definidos como aquellos con mayor número de reviews. Esta visualización nos permite identificar los productos con mayor interacción por parte de los usuarios.

2. Modelo de Machine Learning

A partir de los datos almacenados en la base de datos, hemos querido plantear un sistema de recomendación de artículos para usuarios. Nuestra idea consiste en intentar predecir qué productos le podrían interesar a un usuario basándonos en el comportamiento de otros usuarios y en las valoraciones que ya existen en el sistema.

En este proyecto, la información más útil para construir este sistema de recomendación es la relacionada con las reviews. En concreto, para cada usuario sabemos qué artículos ha valorado y qué nota les ha dado mediante el campo overall. Esto nos permite construir una tabla, conocida como matriz usuario-artículo, en la que cada fila representa un usuario y cada columna representa un producto. En cada posición de esa matriz se almacena la puntuación que ese usuario ha dado a ese artículo. Si un usuario no ha valorado un artículo, esa posición queda vacía.

A partir de esta matriz, una opción que nos parecía razonable era utilizar un enfoque de filtrado colaborativo, basándonos en que si dos usuarios han puntuado de forma parecida varios artículos en común, entonces es probable que tengan gustos similares. Por tanto, si uno de ellos ha consumido un producto que el otro todavía no ha visto, ese producto puede ser una buena recomendación.

Para medir esta similitud entre usuarios, podemos utilizar la correlación de Pearson (como hicimos en apartados anteriores). Esta métrica compara las valoraciones de dos usuarios sobre los artículos que ambos han puntuado. Si normalmente dan notas parecidas, la similitud será alta y si sus gustos son muy distintos, la similitud será baja o incluso negativa. Gracias a esto, para un usuario concreto podemos buscar otros usuarios “parecidos” y, a partir de ellos, recomendar artículos que todavía no haya consumido.

El proceso sería el siguiente: primero seleccionamos un usuario objetivo. Después buscamos otros usuarios con valoraciones similares. A continuación, revisamos los artículos que esos usuarios similares han puntuado pero que el usuario objetivo todavía no ha valorado. Finalmente, ordenamos esos artículos según algún criterio (por ejemplo combinando popularidad y similitud) y los devolvemos como recomendación.

Además del filtrado colaborativo, en este proyecto también podemos aprovechar la información almacenada en MongoDB. En Mongo se guardan campos como reviewText y summary, que contienen texto escrito por los usuarios. Esta información podría utilizarse para enriquecer el sistema de recomendación mediante técnicas de procesamiento de lenguaje natural. Por ejemplo, podríamos analizar las palabras más frecuentes en las reviews positivas de un usuario para detectar qué tipo de características le gustan más. De esta forma, además de recomendar productos porque “a usuarios parecidos les gustaron”, también podríamos recomendar porque “tienen características similares a otros productos bien valorados por ese usuario”.

En resumen, la propuesta consiste en diseñar un recomendador que, dado un usuario, analice sus valoraciones anteriores, busque otros usuarios con gustos similares y le sugiera artículos que todavía no haya consumido.

Usuarios + valoraciones -> Matriz usuario-artículo -> Cálculo de similitud entre usuarios ->Selección de usuarios parecidos -> Búsqueda de artículos no consumidos -> Recomendación final

3. **10 artículos más populares que no ha consumido un usuario**

Hemos implementado un sistema de recomendación sencillo basado en la popularidad. Dado un usuario y una categoría, en primer lugar obtenemos los artículos que el usuario ya ha consumido dentro de dicha categoría. Posteriormente, seleccionamos los artículos más populares (aquellos con mayor número de reviews) y filtramos los que el usuario ya ha consumido. Finalmente, devolvemos los 10 artículos más populares que el usuario todavía no ha visto.